ate

Non - comparison Sorts:

We have seen that comparison based sorts only lead to best $O(n \log n)$ so what if we make a sort that doesn't compare.

We will learn 3 of sorts.

1) Counting Sort
2) Radix Sort
3) Bucket sort.

Counting

* These are also called Linear time sorting

BHATTI

CamScanner

day/date

1) Counting Sort:

```
Counting Sort (A, n, k)
  # n = size of array, k = max element in array.
  {
  C = array of size [0...k] .
  B = array of size n.

for i ⟸ 1 to k
    do  c[i] ⟵ 0

for j ⟸ 1 to n
    do c[A[j]] ⟸ c[A[j]] + 1

for i = 2 to k
    do c[i] = c[i] + c[i-1].

for j = n down to 1
    do B[C[A[j]]] = A[j]
       C[A[j]] ⟸ C[A[j]] - 1.
```

BHATTI

CamScanner

//date

In this we need to first find k which is the maximum element in the array.

e.g in

4	1	3	4	3

we will use this example in full explanation.

k = 4.

Then we make 2 new array.

C array: size k array.

B array: Has result. (size n).

Loop 1:

This loop simply initalizes all the C array with 0.

Loop 2:

This loop counts the amount of time each digit comes in the "A" array.

e.g

4	1	3	4	3

1 comes 1 time

4 comes 2 time

3 each come 2 time.

BHATTI

CamScanner

y/date

After loop 1 our C array:

1	2	3	4
0	0	0	0

C array was of size 4 & then all places initialized to 0.

In loop 2: the index in C corresponding to digit in A will be incremented by one if it exists in A.

eg at j=1, A[1] = 4.

so C[4] = C[4] +1

At end: (C array):

1	2	3	4
1	0	2	2

all counts are precisely ~~stored~~ calculated.

BHATTI

CamScanner

late

loop 3: Cumulative sum.

This simply sums up all the values in C[i] & before it.

at i = 2

C[2] = C[2] + C[1]

C[2] = 0 + 1 = 1.

so now C array:

1	1	2	2

At end of this loop.

C array:

1	1	3	5

BHATTI

CamScanner

ate

Now the benefit of this is we get exact last index positions of each digit:

1	2	3	4
1	1	3	5

for digit 4: count = 5 - 3 = 2
& positions = 5th & 4th.

for digit 3: count = 3 - 1 = 2.
positions 2nd & 3rd.

for digit 2: count = 1 - 1 = 0
so no positions.

for digit 1: count = 1 - 0 = 1
position 1st.

BHATTI

CamScanner

y/date

loop 4:

now we start in reverse because we will be using the index positions in C, that actually has the last positions for each digit.

e.g for 4, it has 5th position but that should be for the ~~2nd~~ 4 second 4 in A array to make it stable. Hence we move in reverse to keep order.

In this simple we take values from A, use the mapping in C & store result in B.

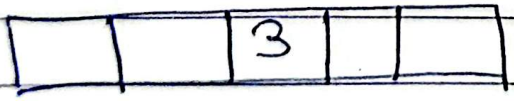
for $j = n = 5$

$B[C[A[5]]] = A[5]$

$B[C[3]] = 3.$

$B[3] = 3$

B array:



now also do $C[A[3]] -= 1$.

so that next time 3 comes it goes in 2.

BHATTI

lay/date

C array now

1	1	2	5

n = 4

B[C[A[4]]] = A[4]

B[C[4]] = 4

B[5] = 4

A array:

1	2	3	4	5
4	1	3	4	3

Stable.

B array =

		3		4

C[A[4]] = C[A[4]] − 1

C array =

1	2	3	4
1	1	2	4

A kendi.

B array

1	2	3	4
1	3	3	4

✓ Sorted.

BHATTI

CamScanner

day/date

IMP:

* Count sort is stable

* But it is NOT inplace.

e.g consider array:

[1 , 3, 2, 1000234]

now, our C would be of size 1000234 which is huge compared to n.

* Current implementation ~~does~~ is only for positive numbers.
(There is also a way for negative numbers. try to find it).

* we can make counting sort ~~inplace~~ use less memory. using hashing !!! (It won't be completly inplace but its k will be equal to n).

BHATTI

CamScanner

date

Complexity

$\Theta(n+k) + \Theta(n)$.

but considering k as small and constant

$\Theta(n) + \Theta(n)$

$\Theta(2n)$

$\Theta(n)$

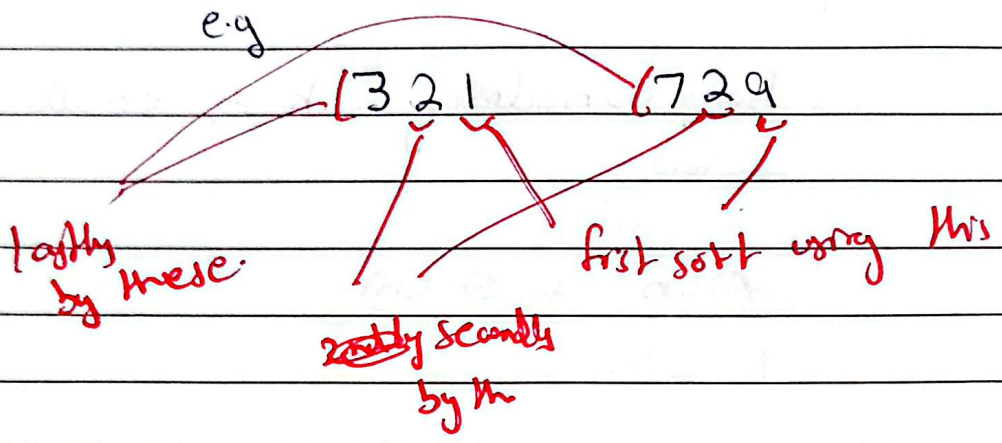
BHATTI

CamScanner

ite

2) Radix Sort

This is basically a digit by digit sort.



In this it is considered a better idea to sort on least significant digits first and at end do most significant instead of other way around.

Radix sort, sorts digits by digit but uses another sort for sorting the digits.

BHATTI

CamScanner

late

Pseudo code:

```
Radix_Sort (A, n)
    # d = number of digits in a number.
    for i=1 to d:

        Use a stable sort to sort
        Array A on digit i.
```

d = number of digits e.g in 321, d=3.

but consider if we get 3 numbers like:

2701 , 93 , 123456

All 3 have different amount of digits.

To fix this we use use padding on smaller numbers.

002701 , 000093 , 123456

now d = 6 ✓

BHATTI

CamScanner

y/date

Example:

Array A:

[329, 457, 657, 839, 436]

First see LSDs:

9, 7, 7, 9, 6.

sort using a stable sort

important to be stable

6, 7, 7, 9, 9.

now array A:

[436, 457, 657, 839, 329]

now next digits:

3, 5, 5, 3, 2

sort

2, 3, 3, 5, 5

BHATTI

CamScanner

date

now array: A

[329, 436, 839, 457, 657]

now next digit:

3, 4, 8, 4, 6

↓ sort

3, 4, 4, 6, 8

Array A:

[329, 436, 457, 657, 839].

SORTED !!!

BHATTI

CamScanner

date

IMP:

Radix Sort can use any other stable sort, it should be stable or it can ruin the array.
e.g insertion sort, counting sort.

- Typically it will use counting sort to keep it completely comparison free

- Dependent upon the sort used inside it it will either sort in place or not

* Does not handle negative number correctly (unless you modify the code to somehow do it).

BHATTI

CamScanner

Complexity:

Consider we used counting sort inside Radix sort.

now counting sort takes $\Theta(n+k)$ & Radix sort runs counting sort d times so:

$$\Theta(d \cdot (n+k))$$

but see that d & k both are constants: so

$$T(n) = \Theta(n)$$

* also see that ~~doooor~~ now the k inside count sort will be maximum 10 because we are doing digit by digit

BHATTI

ate

3) Bucket Sort:

It is for in our input are real numbers (decimals) in range 0 – 1 (inclusive).

Bucket sort works by creating n buckets to divide our interval [0, 1) into equal sub intervals of 1/n size.

- Then traverse the inputs and place them in their appropriate bucket.

- Then sort each bucket using a stable sort like insertion sort or countsort.

- The concatenate the buckets to get results.

Buckets are mainly linked list.

BHATTI

CamScanner

Pseudocode:

```
Bucket Sort (A, n)
    B = array of n empty list        (B: array of linked lists)
    for i = 0 to n
        do insert A[i] into list B[floor(n*A[i])]
    for i = 0 to n-1
        do Insertion sort (B[i])
    Concatenate lists B[0], B[1], -- B[n-1] in order.
```

Example:

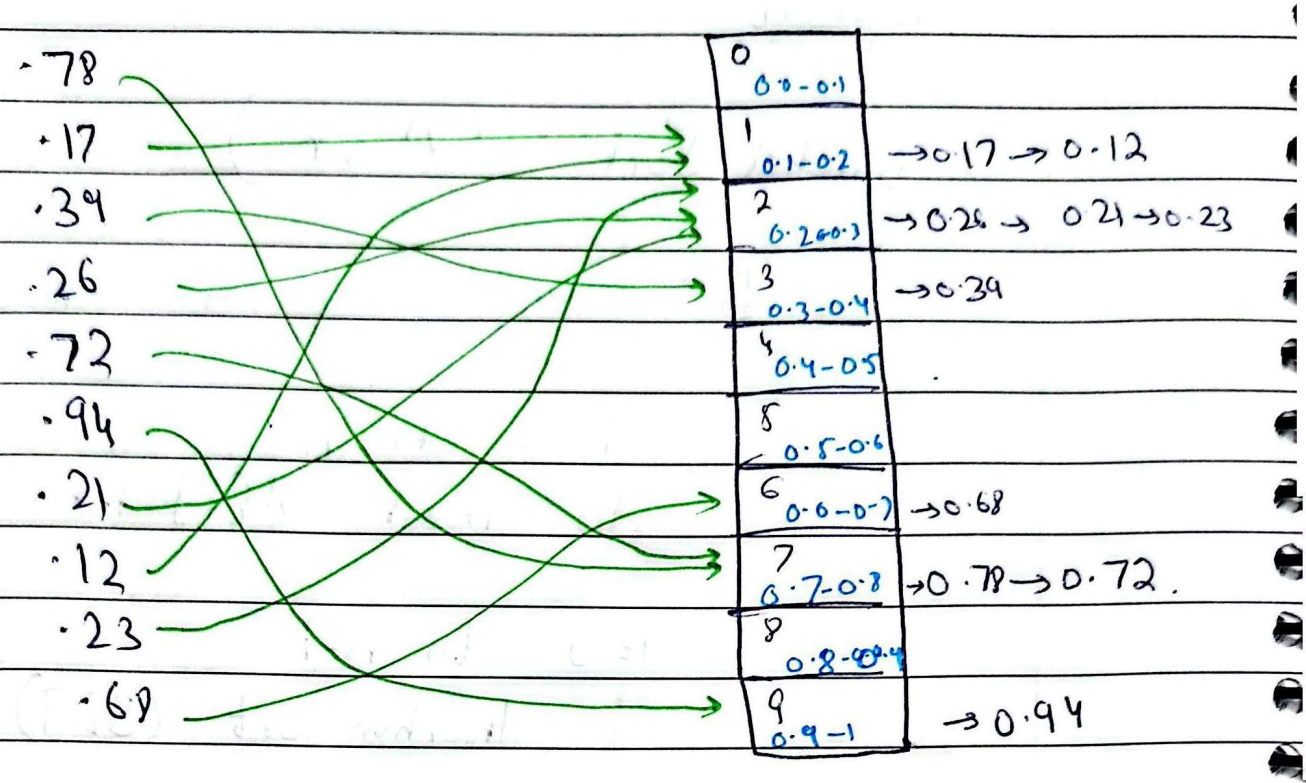
Interval $[0, 1)$:

$A = [.78, .17, .39, .26, .72, .94, .21, .12, .23, .68]$

now $n = 10$, so 10 buckets, each of length 0.1.

BHATTI

CamScanner



now sort each bucket & stably & then concatenate in order.

Sorted!!!

BHATTI

te

Time Complexity:

all loops take $\Theta(n)$ complexity

except:

for i=0 to n-1
    insertion sort.

now worst case would be: all elements go into a single bucket but even in this insertion sort runs $\Theta(n^2)$ but only ONCE, all other buckets are empty so its complexity become $\Theta(n^2) + \Theta(n)$ so $\Theta(n^2)$.

$\Theta(n^2)$ ← worst case.

Average case → $\Theta(n)$

BHATTI

CamScanner